Started on	Wednesday, 20 November 2024, 12:03 AM
State	Finished
Completed on	Sunday, 8 December 2024, 1:05 PM
Time taken	18 days 13 hours
Marks	40.00/40.00
Grade	**100.00** out of 100.00
Name	DHIRU BHAI 730423104017

Question 1

Correct

Mark 5.00 out of 5.00

Write a program with class Rectangle with the data fields width, length and area

The length, width and area are of double type.

Fill in the code wherever required

Do not update the Main class code. It takes care of object creation. You have to update only the Rectangle class

For example:

Input	Result
3.2 2.1 3.2 2.1	setLength:length: 3.2 3.2 setWidth:Width: 2.1 2.1 setLength:length: 3.2 setWidth:Width: 2.1 Exception as expected Matching Rectangles
3.6 3.2 3.6 3.4	setLength:length: 3.6 3.6 setWidth:Width: 3.2 3.2 setLength:length: 3.6 setWidth:Width: 3.4 Exception as expected Non-matching Rectangles
-3.3 4.0 4.0 3.3	setLength:Length must be positive 0.0 setWidth:Width: 4.0 4.0 setLength:length: 4.0 setWidth:Width: 3.3 Exception as expected Non-matching Rectangles
4.0 -3.3 3.3 4.44	setLength:length: 4.0 4.0 setWidthWidth must be positive 0.0 setLength:length: 3.3 setWidth:Width: 4.44 Exception as expected Non-matching Rectangles
-4.0 -3.0 -5.0 -5.5	setLength:Length must be positive 0.0 setWidthWidth must be positive 0.0 setLength:Length must be positive setWidthWidth must be positive Exception as expected Matching Rectangles

Answer: (penalty regime: 0 %)

Reset answer

```
1 import java.util.Scanner;
2 import java.lang.reflect.Field;
3 class Rectangle
```

```
class Rectangle
{
    private double width;
    private double length;
    private double area;
    public void setLength(double length)
    {
        if (length > 0)
        {
            this.length = length;
            System.out.println(Thread.currentThread().getStackTrace()[1].get
        }
        else
        {
            System.out.println(Thread.currentThread().getStackTrace()[1].get
        }
    }
    public void setWidth(double width)
    {
        if (width > 0)
        {
            this.width = width;
            System.out.println(Thread.currentThread().getStackTrace()[1].get
        }
        else
        {
            System.out.println(Thread.currentThread().getStackTrace()[1].get
        }
    }
    public double getLength()
    {
        return length;
    }
    public double getWidth()
    {
        return width;
    }
    public double getArea()
    {
        return area;
    }
    public void findArea()
    {
        if(this.length > 0 || this.width > 0)
            this.area = this.length * this.width;
        else
            this.area = 0;
    }
}
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        double len1 = sc.nextDouble();
        double breadth1 = sc.nextDouble();
        double len2 = sc.nextDouble();
        double breadth2 = sc.nextDouble();
        Rectangle rect1 = new Rectangle();
        Rectangle rect2 = new Rectangle();
        rect1.setLength(len1);
        System.out.println(rect1.getLength());
        rect1.setWidth(breadth1);
        System.out.println(rect1.getWidth());
        rect1.findArea();
        rect2.setLength(len2);
        rect2.setWidth(breadth2);
        rect2.findArea();
        try
        {
            Field lengthField = Rectangle.class.getDeclaredField("length");
            System.out.println((double) lengthField.get(rect1));
        }
        catch (NoSuchFieldException | IllegalAccessException e)
```

```
        catch (NoSuchFieldException | IllegalAccessException e)
        {
            System.out.println("Exception as expected");
        }
        if (rect1.getArea() == rect2.getArea())
        {
            System.out.println("Matching Rectangles");
        }
        else
        {
            System.out.println("Non-matching Rectangles");
        }
    }
}
```

	Input	Expected	Got	
✔	3.2 2.1 3.2 2.1	setLength:length: 3.2 3.2 setWidth:Width: 2.1 2.1 setLength:length: 3.2 setWidth:Width: 2.1 Exception as expected Matching Rectangles	setLength:length: 3.2 3.2 setWidth:Width: 2.1 2.1 setLength:length: 3.2 setWidth:Width: 2.1 Exception as expected Matching Rectangles	✔
✔	3.6 3.2 3.6 3.4	setLength:length: 3.6 3.6 setWidth:Width: 3.2 3.2 setLength:length: 3.6 setWidth:Width: 3.4 Exception as expected Non-matching Rectangles	setLength:length: 3.6 3.6 setWidth:Width: 3.2 3.2 setLength:length: 3.6 setWidth:Width: 3.4 Exception as expected Non-matching Rectangles	✔
✔	-3.3 4.0 4.0 3.3	setLength:Length must be positive 0.0 setWidth:Width: 4.0 4.0 setLength:length: 4.0 setWidth:Width: 3.3 Exception as expected Non-matching Rectangles	setLength:Length must be positive 0.0 setWidth:Width: 4.0 4.0 setLength:length: 4.0 setWidth:Width: 3.3 Exception as expected Non-matching Rectangles	✔
✔	4.0 -3.3 3.3 4.44	setLength:length: 4.0 4.0 setWidthWidth must be positive 0.0 setLength:length: 3.3 setWidth:Width: 4.44 Exception as expected Non-matching Rectangles	setLength:length: 4.0 4.0 setWidthWidth must be positive 0.0 setLength:length: 3.3 setWidth:Width: 4.44 Exception as expected Non-matching Rectangles	✔
✔	-4.0 -3.0 -5.0 -5.5	setLength:Length must be positive 0.0 setWidthWidth must be positive 0.0 setLength:Length must be positive setWidthWidth must be positive Exception as expected Matching Rectangles	setLength:Length must be positive 0.0 setWidthWidth must be positive 0.0 setLength:Length must be positive setWidthWidth must be positive Exception as expected Matching Rectangles	✔

Passed all tests! ✔

▸ **Show/hide question author's solution (Java)**

Question 2

Correct

Mark 5.00 out of 5.00

Write a program with class Circle with the data fields radius and area

The fields radius and area are of double type.

Fill in the code wherever required

Do not update the Main class code. It takes care of object creation. You have to update only the Rectangle class

For example:

Input	Result
3.2 2.1 3.2 2.1	setRadius:radius: 3.2 3.2 setRadius:radius: 2.1 Exception as expected Non-matching Circles
3.6 3.2 3.6 3.4	setRadius:radius: 3.6 3.6 setRadius:radius: 3.2 Exception as expected Non-matching Circles
-3.3 4.0 4.0 3.3	setRadius:radius must be positive 0.0 setRadius:radius: 4.0 Exception as expected Non-matching Circles
4.0 -3.3 3.3 4.44	setRadius:radius: 4.0 4.0 setRadius:radius must be positive Exception as expected Non-matching Circles
-4.0 -3.0 -5.0 -5.5	setRadius:radius must be positive 0.0 setRadius:radius must be positive Exception as expected Matching Circles

Answer: (penalty regime: 0 %)

Reset answer

```
import java.util.Scanner;
import java.lang.reflect.Field;
class Circle
{
    private double radius;
    private double area;
    public void setRadius(double radius)
    {
        if (radius > 0)
        {
            this.radius = radius;
            System.out.println(Thread.currentThread().getStackTrace()[1].get
        } else {
            System.out.println(Thread.currentThread().getStackTrace()[1].get
        }
    }
    public double getRadius()
    {
        return radius;
```

```
    }
    public double getArea()
    {
        return area;
    }
    public void findArea()
    {
        if(this.radius > 0)
            this.area = Math.PI * radius * radius;
        else
            this.area = 0;
    }

}
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        double r1 = sc.nextDouble();
        double r2 = sc.nextDouble();
        Circle c1 = new Circle();
        Circle c2 = new Circle();
        c1.setRadius(r1);
        System.out.println(c1.getRadius());
        c1.findArea();
        c2.setRadius(r2);
        c2.findArea();
        try
        {
            Field radiusField = Circle.class.getDeclaredField("radius");
            System.out.println((double) radiusField.get(c1));
        }
        catch (NoSuchFieldException | IllegalAccessException e)
        {
            System.out.println("Exception as expected");
        }
        if (c1.getArea() == c2.getArea()) {
            System.out.println("Matching Circles");
        } else {
            System.out.println("Non-matching Circles");
        }
    }
}
```

	Input	Expected	Got	
✔	3.2 2.1 3.2 2.1	setRadius:radius: 3.2 3.2 setRadius:radius: 2.1 Exception as expected Non-matching Circles	setRadius:radius: 3.2 3.2 setRadius:radius: 2.1 Exception as expected Non-matching Circles	✔
✔	3.6 3.2 3.6 3.4	setRadius:radius: 3.6 3.6 setRadius:radius: 3.2 Exception as expected Non-matching Circles	setRadius:radius: 3.6 3.6 setRadius:radius: 3.2 Exception as expected Non-matching Circles	✔
✔	-3.3 4.0 4.0 3.3	setRadius:radius must be positive 0.0 setRadius:radius: 4.0 Exception as expected Non-matching Circles	setRadius:radius must be positive 0.0 setRadius:radius: 4.0 Exception as expected Non-matching Circles	✔

	Input	Expected	Got	
✔	4.0 -3.3 3.3 4.44	setRadius:radius: 4.0 4.0 setRadius:radius must be positive Exception as expected Non-matching Circles	setRadius:radius: 4.0 4.0 setRadius:radius must be positive Exception as expected Non-matching Circles	✔
✔	-4.0 -3.0 -5.0 -5.5	setRadius:radius must be positive 0.0 setRadius:radius must be positive Exception as expected Matching Circles	setRadius:radius must be positive 0.0 setRadius:radius must be positive Exception as expected Matching Circles	✔

Passed all tests! ✔

▸ **Show/hide question author's solution (Java)**

Correct

Marks for this submission: 5.00/5.00.

Question 3

Correct

Mark 10.00 out of 10.00

Create a class called **Author** which should be designed as follows:

- Three private instance variables: name (String), email (String), and gender (char of either 'm' or 'f').
- One constructor to initialize the name, email and gender with the given values.

And, a class called **Book** which should be designed as follows:

- Four private instance variables: name (String), author (of the class Author you have just created), price (double), and qtyInStock (int). Assuming that each book is written by one author.
- One constructor which constructs an instance with the values given.
- Getters and setters: getName(), getAuthor(), getPrice(), setPrice(), getQtyInStock(), setQtyInStock().

Again there is no setter for name and author.

Write main method in class Book (which uses the Author class written earlier) and perform the following operations.

- Print the book name, price and qtyInStock from a Book instance. (Hint: aBook.getName())
- After obtaining the "Author" object, print the Author (name, email & gender) of the book.

For example:

Input	Result
C Programming Dennis Ritchie dr@gmail.com M 750.28 7	Book Name: C Programming Author Name: Dennis Ritchie Author MailId: dr@gmail.com Author Gender: M Book Price: 750.28 Quantity in Stock: 7
Java The Complete Reference Herbert Schildt hs@gmail.com M 975.54 24	Book Name: Java The Complete Reference Author Name: Herbert Schildt Author MailId: hs@gmail.com Author Gender: M Book Price: 975.54 Quantity in Stock: 24

Answer: (penalty regime: 0 %)

Reset answer

```
import java.util.*;
class Author
{
    public String Name;
    public String MailId;
    public char Gender;
    Author(String Name,String MailId,char Gender)
    {
        this.Name=Name;
        this.MailId=MailId;
        this.Gender=Gender;
    }
    public String getName()
    {
        return Name;
```

```
    }
    public String getMailId()
    {
        return MailId;
    }
    public char getGender()
    {
        return Gender;
    }
}
class Book
{
    private String Name;
    private Author author;
    private double price;
    private int qtyInstock;
    Book(String Name,Author author,double price,int qtyInstock)
    {
        this.Name=Name;
        this.author=author;
        this.price=price;
        this.qtyInstock=qtyInstock;
    }
    public String getName()
    {
        return Name;
    }
    public Author getAuthor()
    {
        return author;
    }
    public double getPrice()
    {
        return price;
    }
    public int getqtyInStock()
    {
        return qtyInstock;
    }
}
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String bookName = sc.nextLine();
        String authorName = sc.nextLine();
        String authorMailID = sc.nextLine();
        char gender = sc.next().charAt(0);
        double bookPrice = sc.nextDouble();
        int quantity = sc.nextInt();
        Book bk=new Book(bookName,new Author(authorName,authorMailID,gender)
        System.out.println("Book Name: "+bk.getName());
        System.out.println("Author Name: "+bk.getAuthor().Name);
        System.out.println("Author MailId: "+bk.getAuthor().MailId);
        System.out.println("Author Gender: "+bk.getAuthor().Gender);
        System.out.println("Book Price: "+bk.getPrice());
        System.out.println("Quantity in Stock: "+bk.getqtyInStock());
    }
}
```

	Input	Expected	Got	
✔	C Programming Dennis Ritchie dr@gmail.com M 750.28 7	Book Name: C Programming Author Name: Dennis Ritchie Author MailId: dr@gmail.com Author Gender: M Book Price: 750.28 Quantity in Stock: 7	Book Name: C Programming Author Name: Dennis Ritchie Author MailId: dr@gmail.com Author Gender: M Book Price: 750.28 Quantity in Stock: 7	✔
✔	Java The Complete Reference Herbert Schildt hs@gmail.com M 975.54 24	Book Name: Java The Complete Reference Author Name: Herbert Schildt Author MailId: hs@gmail.com Author Gender: M Book Price: 975.54 Quantity in Stock: 24	Book Name: Java The Complete Reference Author Name: Herbert Schildt Author MailId: hs@gmail.com Author Gender: M Book Price: 975.54 Quantity in Stock: 24	✔

Passed all tests! ✔

▸ **Show/hide question author's solution (Java)**

Correct

Marks for this submission: 10.00/10.00.

Question 4

Correct

Mark 20.00 out of 20.00

Encapsulation in a Bank Account System

You are required to write a program that models a **Bank Account** system using Java. The `BankAccount` class will have private fields for the account number, account holder's name, and balance. The program should simulate deposit and withdrawal operations while ensuring that the account balance cannot go negative.

Specifications:

1. **BankAccount Class**:
   - **Private Fields**:
     - `String accountNumber` (the account number)
     - `String accountHolderName` (the name of the account holder)
     - `double balance` (the balance in the account)
   - **Setter Methods**:
     - `setAccountNumber(String accountNumber)`: Set the account number (ensure the account number is not empty).
     - `setAccountHolderName(String accountHolderName)`: Set the account holder's name (ensure the name is not empty).
     - `setBalance(double balance)`: Set the balance, but only if the balance is non-negative. If a negative balance is provided, print a message saying "Balance cannot be negative".
   - **Getter Methods**:
     - `getAccountNumber()`: Return the account number.
     - `getAccountHolderName()`: Return the account holder's name.
     - `getBalance()`: Return the current balance of the account.
   - **Methods for Transactions**:
     - `deposit(double amount)`: Deposit an amount into the account. If the amount is positive, add it to the balance and print the updated balance. If the amount is zero or negative, print a message saying "Deposit amount must be positive".
     - `withdraw(double amount)`: Withdraw an amount from the account. If the withdrawal amount is positive and does not exceed the current balance, subtract it from the balance and print the updated balance. If the withdrawal amount is greater than the balance or is non-positive, print an appropriate error message.
2. **Main Class**:
   - The `main` method should:
     - Create two `BankAccount` objects.
     - Allow the user to input account details (account number, account holder name, and initial balance) for both accounts.
     - Perform a series of operations such as deposit and withdrawal for both accounts and display the updated balances.

Constraints:

- Ensure proper encapsulation by making sure that all fields in the `BankAccount` class are `private` and that the balance cannot be set to a negative value.

For example:

Input	Result
123456 John Doe 1000 987654 Alice Smith 500 200 300 600	Depositing 200.0 to account 123456. New balance: 1200.0 Withdrawing 300.0 from account 123456. New balance: 900.0 Attempted withdrawal of 600.0 from account 987654 failed. Insufficient funds. Account 1 Details: Account Number: 123456 Account Holder: John Doe Balance: 900.0 Account 2 Details: Account Number: 987654 Account Holder: Alice Smith Balance: 500.0

Input	Result
415634 David Smith 1000 987654 Alice Smith 500 200 300 -600	Depositing 200.0 to account 415634. New balance: 1200.0 Withdrawing 300.0 from account 415634. New balance: 900.0 Withdrawal amount must be positive. Account 1 Details: Account Number: 415634 Account Holder: David Smith Balance: 900.0 Account 2 Details: Account Number: 987654 Account Holder: Alice Smith Balance: 500.0

Answer: (penalty regime: 0 %)

Reset answer

```
import java.util.Scanner;
class BankAccount
{
    private String accountNumber;
    private String accountHolderName;
    private double balance;
    public void setAccountNumber(String accountNumber)
    {
        this.accountNumber = accountNumber;
    }
    public void setAccountHolderName(String accountHolderName)
    {
        this.accountHolderName = accountHolderName;
    }
    public void setBalance(double balance)
    {
        this.balance = balance;
    }
    public String getAccountNumber()
    {
        return accountNumber;
    }
    public String getAccountHolderName()
    {
        return accountHolderName;
    }
    public double getBalance()
    {
        return balance;
    }
    public void deposit(double amount)
    {
        if (amount > 0)
        {
            balance += amount;
            System.out.println("Depositing " + amount + " to account " + acc
        }
        else
        {
            System.out.println("Deposit amount must be positive.");
        }
    }
    public void withdraw(double amount)
    {
        if (amount > 0 && amount <= balance)
        {
            balance -= amount;
            System.out.println("Withdrawing " + amount + " from account " +
        }
        else if (amount > balance)
```

```
        {
            System.out.println("Attempted withdrawal of " + amount + " from
        }
        else
        {
            System.out.println("Withdrawal amount must be positive.");
        }
    }
}
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        BankAccount account1 = new BankAccount();
        BankAccount account2 = new BankAccount();
        account1.setAccountNumber(sc.nextLine());
        account1.setAccountHolderName(sc.nextLine());
        account1.setBalance(sc.nextDouble());
        sc.nextLine();
        account2.setAccountNumber(sc.nextLine());
        account2.setAccountHolderName(sc.nextLine());
        account2.setBalance(sc.nextDouble());
        account1.deposit(sc.nextDouble());
        account1.withdraw(sc.nextDouble());
        account2.withdraw(sc.nextDouble());
        System.out.println("\nAccount 1 Details:");
        System.out.println("Account Number: " + account1.getAccountNumber())
        System.out.println("Account Holder: " + account1.getAccountHolderNam
        System.out.println("Balance: " + account1.getBalance());
        System.out.println("\nAccount 2 Details:");
        System.out.println("Account Number: " + account2.getAccountNumber())
        System.out.println("Account Holder: " + account2.getAccountHolderNam
        System.out.println("Balance: " + account2.getBalance());
    }
}
```

	Input	Expected	Got	
✔	123456 John Doe 1000 987654 Alice Smith 500 200 300 600	Depositing 200.0 to account 123456. New balance: 1200.0 Withdrawing 300.0 from account 123456. New balance: 900.0 Attempted withdrawal of 600.0 from account 987654 failed. Insufficient funds. Account 1 Details: Account Number: 123456 Account Holder: John Doe Balance: 900.0 Account 2 Details: Account Number: 987654 Account Holder: Alice Smith Balance: 500.0	Depositing 200.0 to account 123456. New balance: 1200.0 Withdrawing 300.0 from account 123456. New balance: 900.0 Attempted withdrawal of 600.0 from account 987654 failed. Insufficient funds. Account 1 Details: Account Number: 123456 Account Holder: John Doe Balance: 900.0 Account 2 Details: Account Number: 987654 Account Holder: Alice Smith Balance: 500.0	✔

	Input	Expected	Got	
✔	415634 David Smith 1000 987654 Alice Smith 500 200 300 -600	Depositing 200.0 to account 415634. New balance: 1200.0 Withdrawing 300.0 from account 415634. New balance: 900.0 Withdrawal amount must be positive. Account 1 Details: Account Number: 415634 Account Holder: David Smith Balance: 900.0 Account 2 Details: Account Number: 987654 Account Holder: Alice Smith Balance: 500.0	Depositing 200.0 to account 415634. New balance: 1200.0 Withdrawing 300.0 from account 415634. New balance: 900.0 Withdrawal amount must be positive. Account 1 Details: Account Number: 415634 Account Holder: David Smith Balance: 900.0 Account 2 Details: Account Number: 987654 Account Holder: Alice Smith Balance: 500.0	✔

Passed all tests! ✔

▸ **Show/hide question author's solution (Java)**

Correct

Marks for this submission: 20.00/20.00.